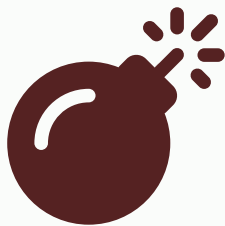


Architecture

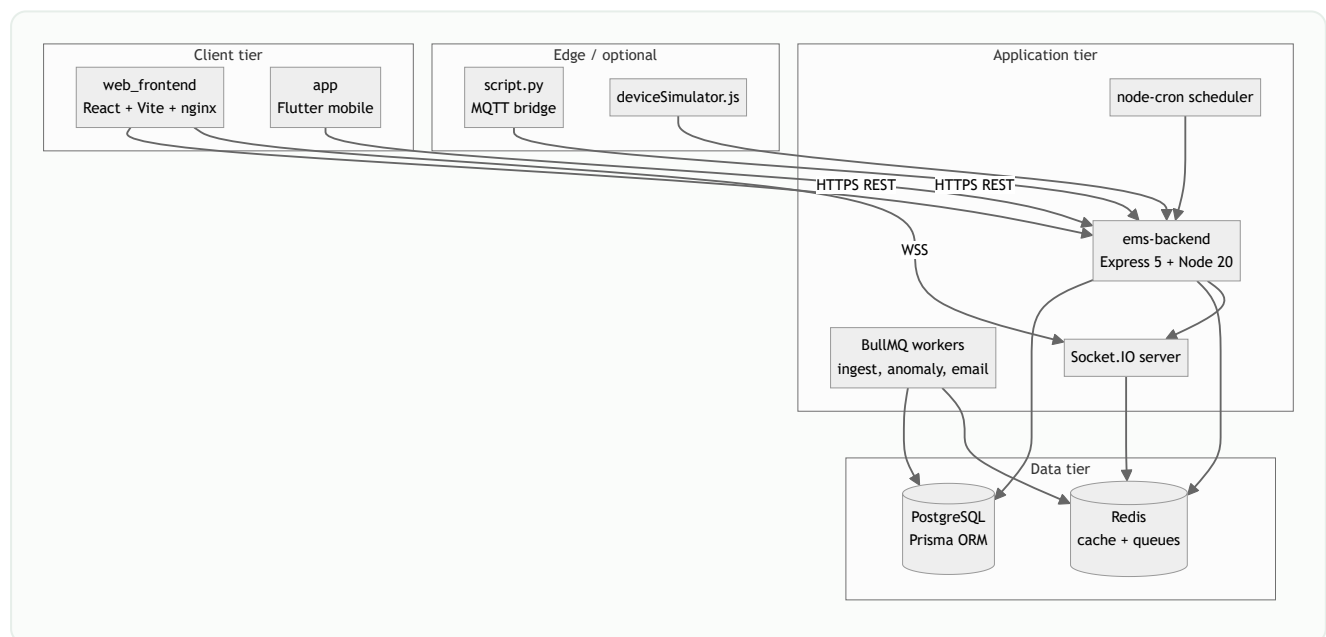
Technical architecture of the Smart AgriTech EMS platform: components, data flow, security, and deployment topology.

Logical architecture

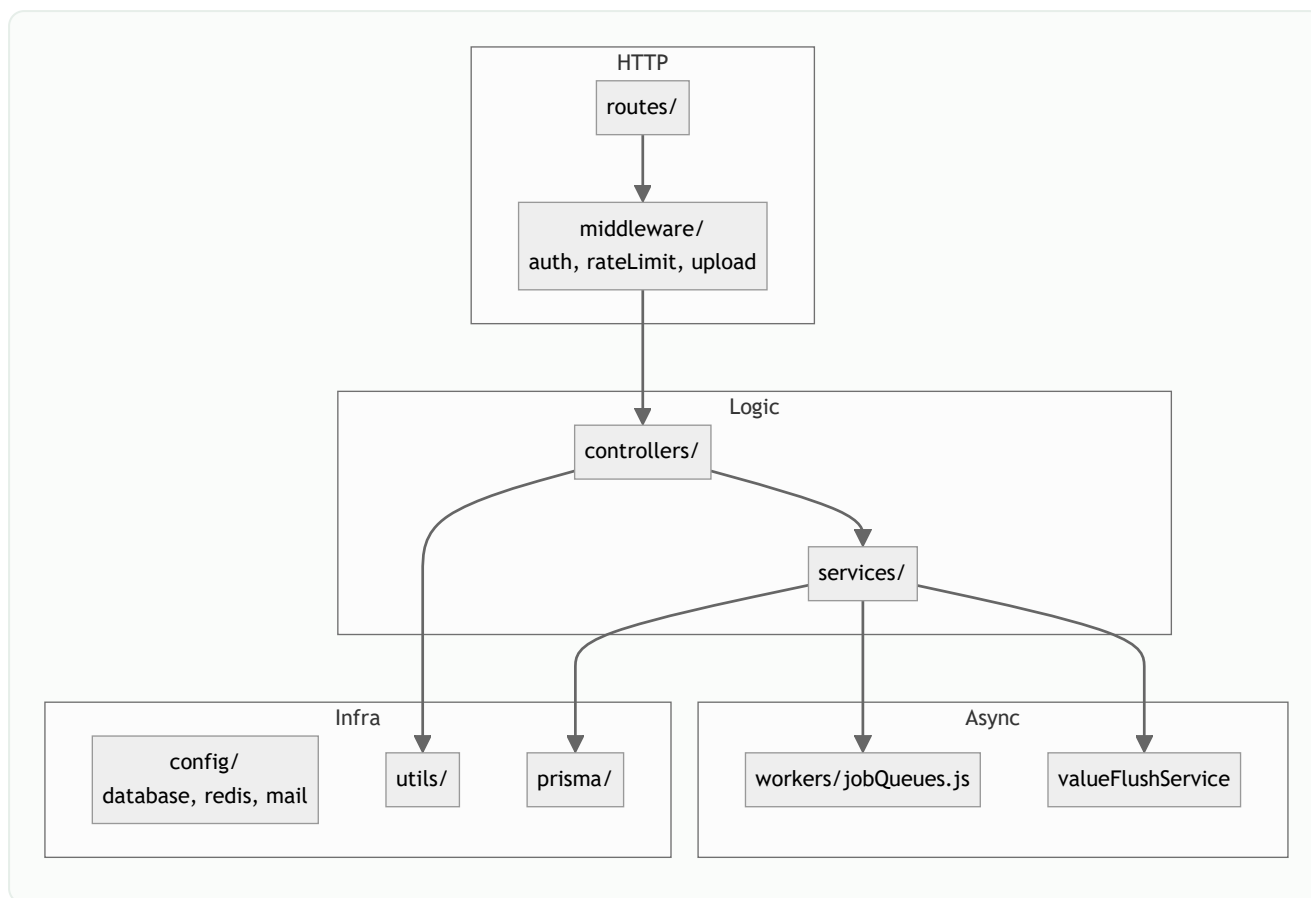


Syntax error in text
mermaid version 10.9.6

Container diagram

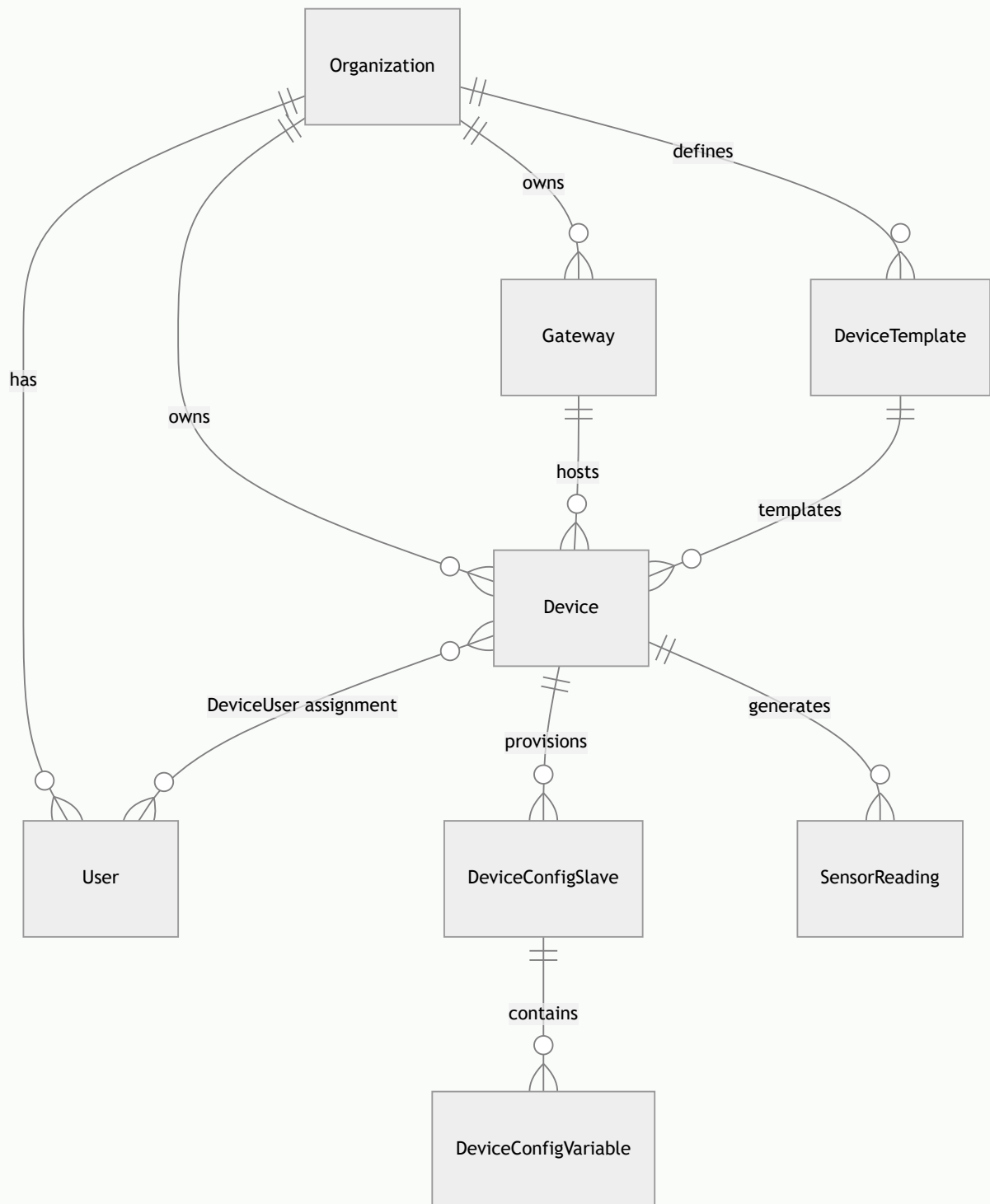


Backend internal layers



Layer	Responsibility
Routes	URL mapping, mount order (<code>/api/ingest</code> before <code>/api</code>)
Middleware	JWT <code>protect</code> , role <code>authorize</code> , rate limits, multer uploads
Controllers	Request validation, org scoping, HTTP responses
Services	Business logic: ingest, anomalies, notifications, scheduler
Workers	BullMQ: batched ingest, anomaly checks, email, device delete
Prisma	40+ models, migrations, type-safe queries

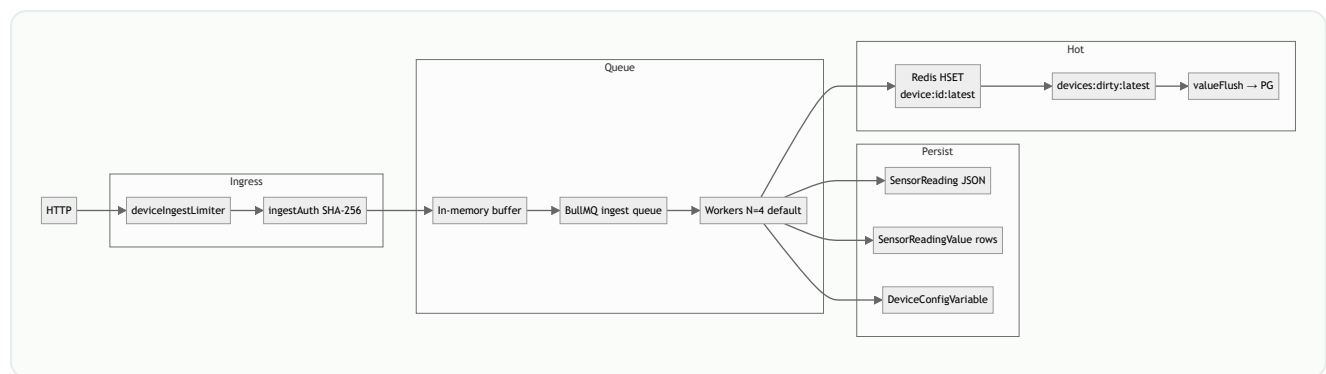
Multi-tenancy model



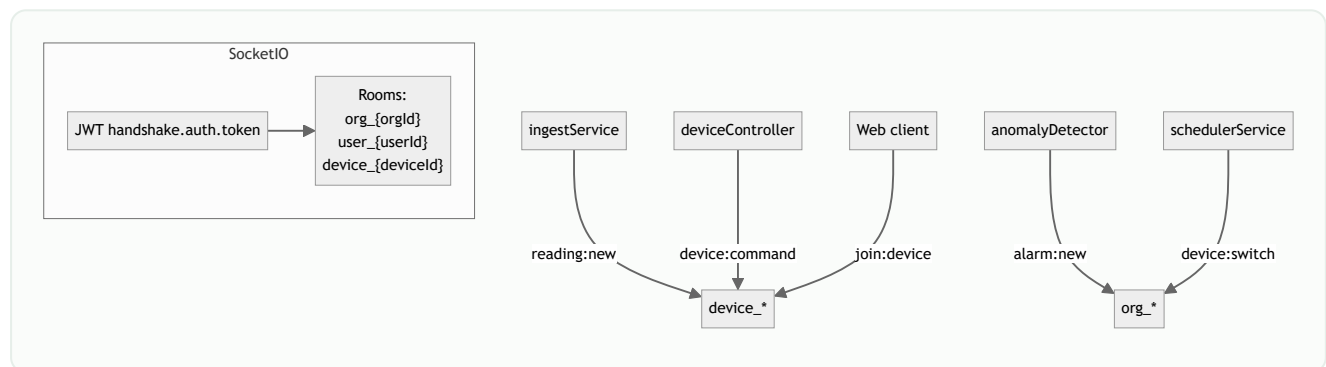
Isolation rules:

- `ORG_ADMIN` and `USER` queries scoped to `user.organizationId`.
- `USER` device list filtered through `DeviceUser` join.
- `SUPER_ADMIN` can query any org (optional `organizationId` filter).

Ingest architecture (production path)

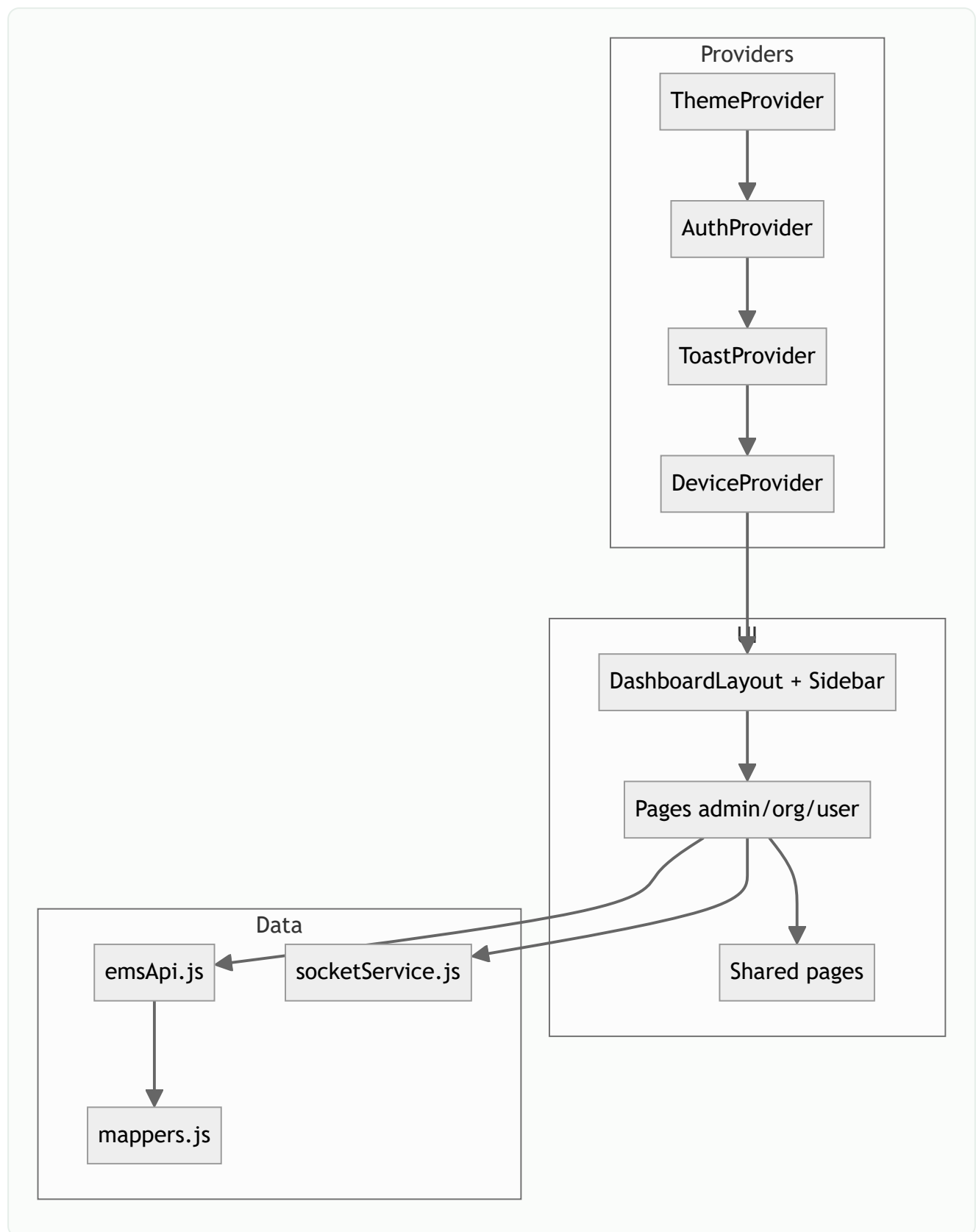


Real-time architecture



Optional **Redis adapter** (`@socket.io/redis-adapter`) enables horizontal scaling of API instances.

Web frontend architecture



Build output: static SPA served by **nginx** (Docker) with API URL baked at build time (`VITE_API_URL` , `VITE_SOCKET_URL`).

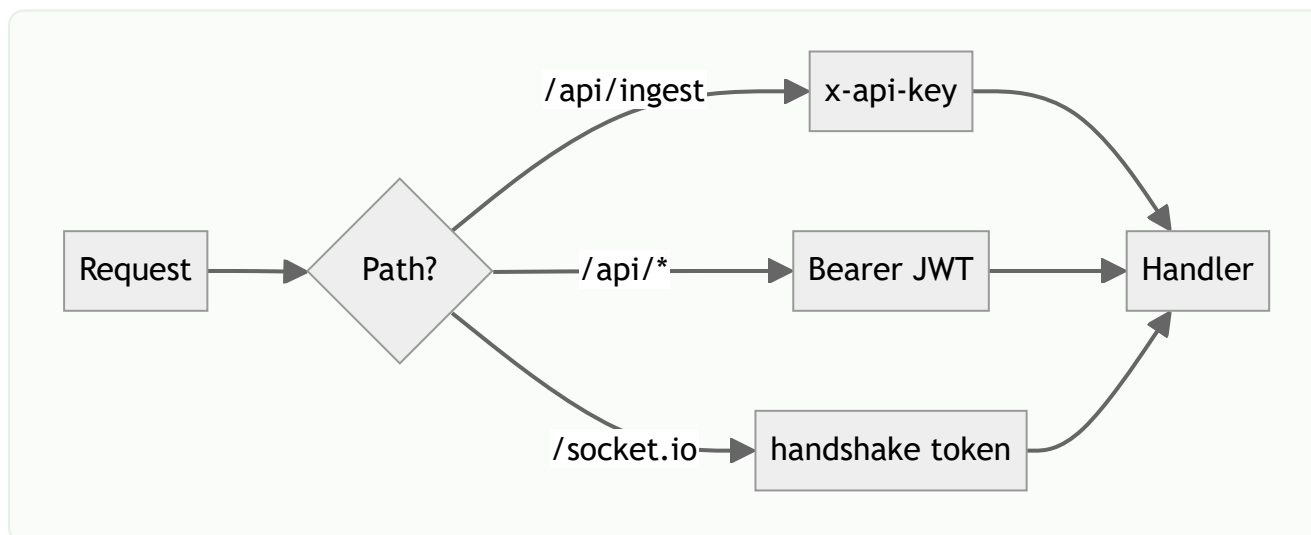
Mobile app (Flutter)

Located in `app/` — companion client for the same REST API:

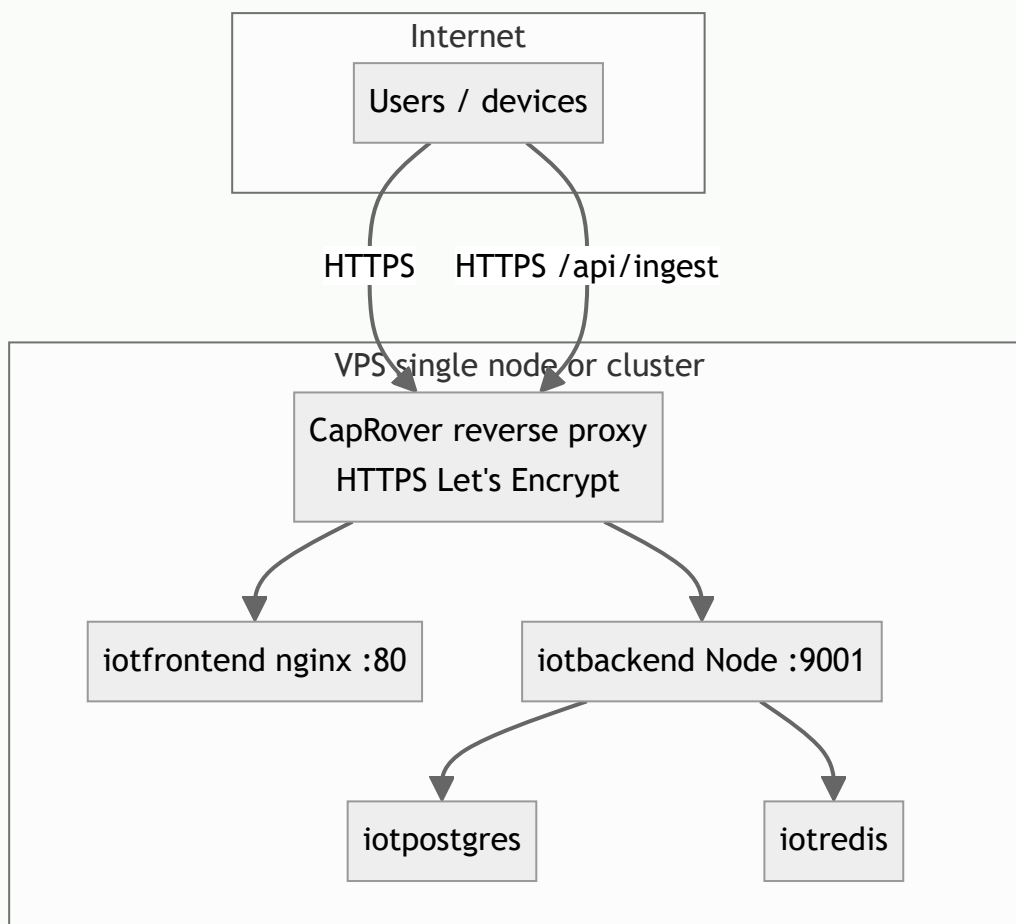
- Auth, device list, dashboards (role-aligned with backend).
- Shares backend JWT auth model.
- Deployed separately (Android/iOS builds), not part of CapRover web stack.

Security architecture

Concern	Implementation
Authentication	JWT access (15m default) + refresh tokens (hashed in DB)
Authorization	Role middleware + per-controller org/device checks
Ingest auth	<code>x-api-key</code> header — global <code>INGEST_API_KEY</code> or per-device hashed key
Rate limiting	<code>express-rate-limit</code> + Redis store when available
CORS	<code>CLIENT_URL</code> comma-separated origins
Cookies	Optional httpOnly token cookies in auth flow
Helmet	Security headers on API
Uploads	Cloudinary for icons/products (optional)



Production deployment topology (CapRover)



Internal DNS (CapRover): `srv-captain--{appName}` for service-to-service URLs.

Observability

Endpoint	Purpose
<code>GET /health</code>	Liveness: DB, Redis, ingest mode
<code>GET /metrics</code>	Prometheus text format counters
App logs	Docker/CapRover container logs
Prisma logs	Opt-in <code>PRISMA_LOG_QUERIES=true</code>

Scalability considerations

Bottleneck	Mitigation
High ingest rate	Redis + BullMQ batching; increase <code>INGEST_WORKER_CONCURRENCY</code>
Dashboard reads	Redis latest cache; <code>SKIP_PG_CURRENT_VALUE</code> ; response cache on dashboard summary
Socket fan-out	Redis Socket.IO adapter; multiple backend replicas behind CapRover
Time-series growth	TimescaleDB hypertables; archive scripts in <code>scripts/archive-cold-data</code>
DB connections	PgBouncer (documented in <code>.env.example</code>); pool tuning <code>DB_POOL_*</code>

Related documents

- [System flows](#)
- [Tech stack](#)
- [Backend details](#)
- [Deployment](#)